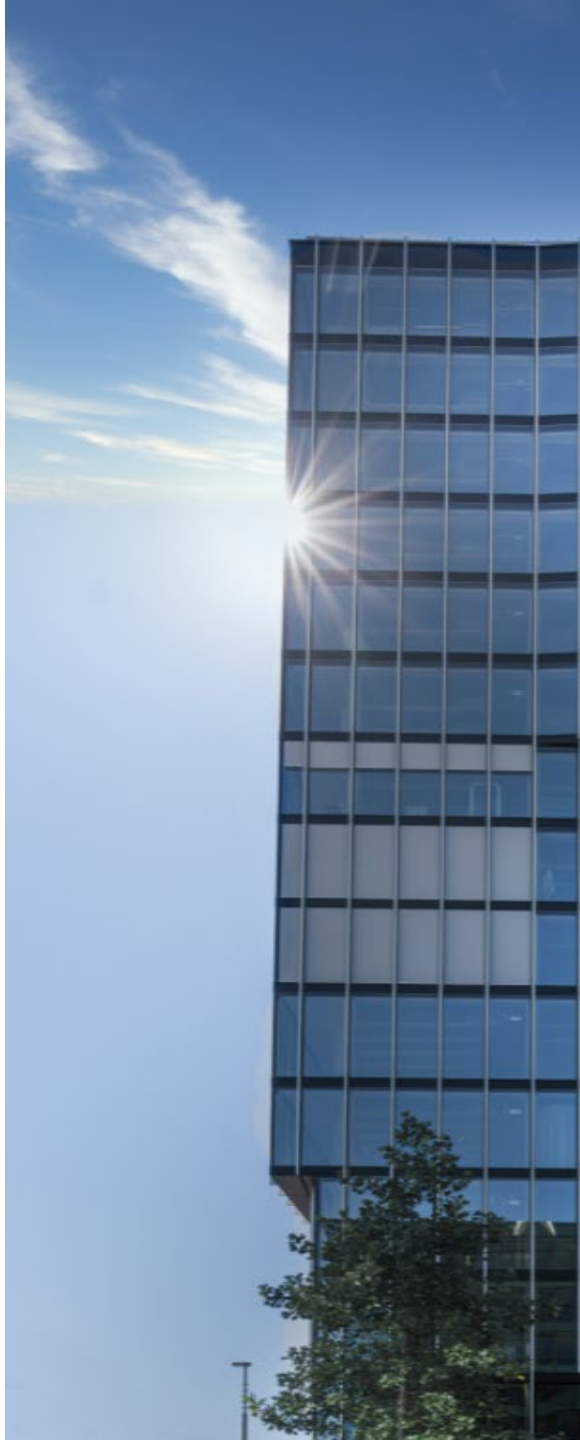
HSLU Hochschule Luzern

Objektorientierte Programmierung

`equals()`, `hashcode()` und `Comparable`

Roland Gisler



FH Zentralschweiz

Inhalt

- Das Gleiche ist nicht dasselbe!
- Verschiedene Arten von Gleichheit.
- `equals()`-Contract und Implementation.
- `hashCode()`-Contract und Implementation.
- Testen von `equals()` und `hashCode()`.
- Vergleich von Objekten: `Comparable<T>`.
- Zusammenfassung

Lernziele

- Sie kennen die unterschiedlichen Arten von «Gleichheit».
- Sie verstehen den Inhalt und die Notwendigkeit des **`equals`**- und **`hashcode`**-Contracts.
- Sie sind in der Lage adäquate Implementationen von **`equals()`** und **`hashcode()`** vorzunehmen, zu beurteilen und zu testen.
- Sie können die natürliche Ordnung mit **`Comparable`** festlegen.
- Sie können einfache **`Comparator`**en für spezielle Ordnungen implementieren.

Das Gleiche ist nicht dasselbe!

Das Gleiche ist nicht dasselbe!

- Worin besteht der Unterschied zwischen den folgenden Aussagen:
  - Jemand benutzt **eine** Zahnbürste.
  - Jemand benutzt die **gleiche** Zahnbürste (wie Sie).
  - Jemand benutzt **dieselbe** Zahnbürste (wie Sie).
- Wenn die Polizei jemanden sucht:
  1. Signalement: «ca. 1.80m gross, dunkelblond, Brillenträger, spricht Dialekt,…»
  2. Gesucht wird: «Roland Gisler, von Luzern, gibt sich als Dozent aus!»
  3. Letztlich erfolgt die absolut eindeutige Identifikation
     z.B. anhand eines Fingerabdrucks oder über die DNA/Genetik.
- Die benötigte «Gleichheit» (strenge) ist also immer auch abhängig vom ➔Kontext!

Welche Art von Gleichheit brauchen wir wann?

Diese Frage ist nicht immer ganz einfach zu beantworten:

- Gibt es gleiche Zahlen?
  - **Ja**, falls sie in ihrem Wert übereinstimmen.
- Gibt es gleiche Farben?
  - **Ja**, falls die drei RGB-Werte (**rot**/**grün**/**blau**)* übereinstimmen.
- Gibt es gleiche Studierende?
  - **Nein**, jede/r Studierende ist einzigartig, *aber…*

    Ja, es gibt mehrere Informatik-Studierende mit dunklen Haaren.
- Gibt es gleiche Schulen?
  - **Ja**, es gibt mehrere Fachhochschulen (FHs), *aber…*

    Nein, die **HSLU** ist absolut **einzigartig**!

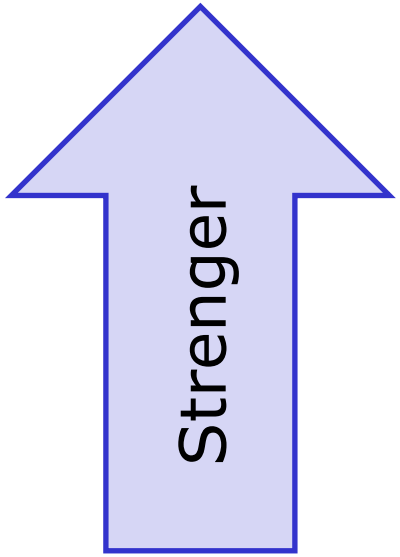
➔Teilweise kann man philosophieren (z.B. über Farben streiten)!

*Welche Farben sehen Sie hier? ☺

Drei verschiedene Arten von «Gleichheit» von Objekten

Man unterscheidet in der Programmierung zwischen **drei** verschieden strengen Arten von «Gleichheit», die je nach Kontext und Anforderungen zum Einsatz kommen:

- **Identität** (Entity Types)
  - Identisches, selbes Objekt.
- **Wertegleichheit** (Value Types)
  - Objekt mit wertgleichen Attributen.
- **Typgleichheit** (Class Types)
  - Objekt von der gleichen Klasse.



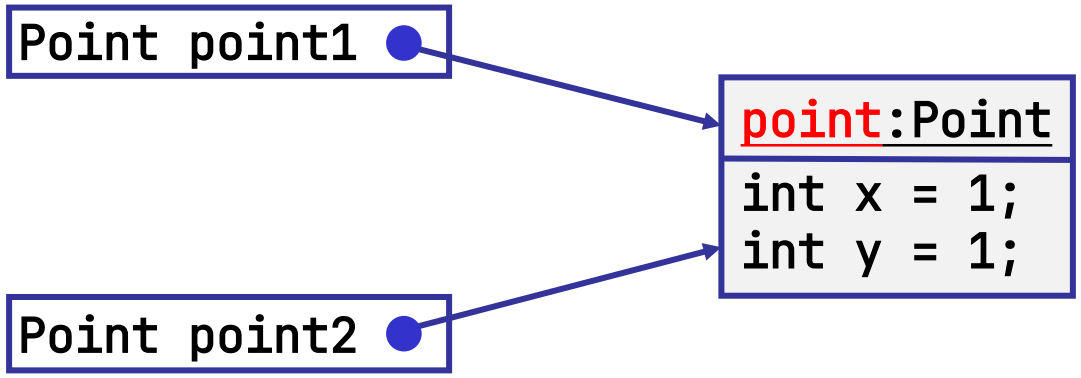
➔ Es besteht somit eine implizite Reihenfolge bezüglich der «Strenge», mit der eine potenzielle Gleichheit betrachtet wird.

Identität (Entity Types)

- Die absolute **Identität** eines Objektes steht im Vordergrund.
- Gleichheit ist **nur dann** gegeben, wenn es sich um (ein und) das**selbe** Objekt handelt: Die **Referenz** ist identisch.

```
Point point1 = new Point(1,1);
Point point2 = point1;
```

➔ `point1` und `point2` sind ein und der**selbe** Punkt!

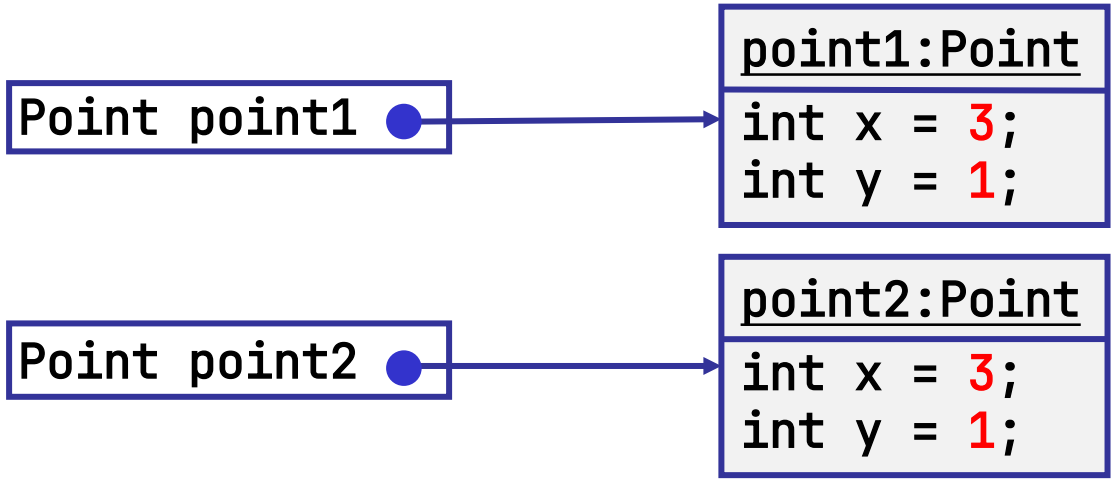


Wertegleichheit (Value Types)

- Der **Zustand** eines Objektes steht im Vordergrund.
  - Attribute enthalten **Werte,** welchen den Zustand repräsentieren.
- Gleichheit ist dann gegeben, wenn ausgewählte (!) Attribute übereinstimmen: Ausgewählte **Werte** sind somit gleich.

```
Point point1 = new Point(3,1);
Point point2 = new Point(3,1);
```

➔ `point1` und `point2` sind zwei Objekte, aber der **gleiche** Punkt!



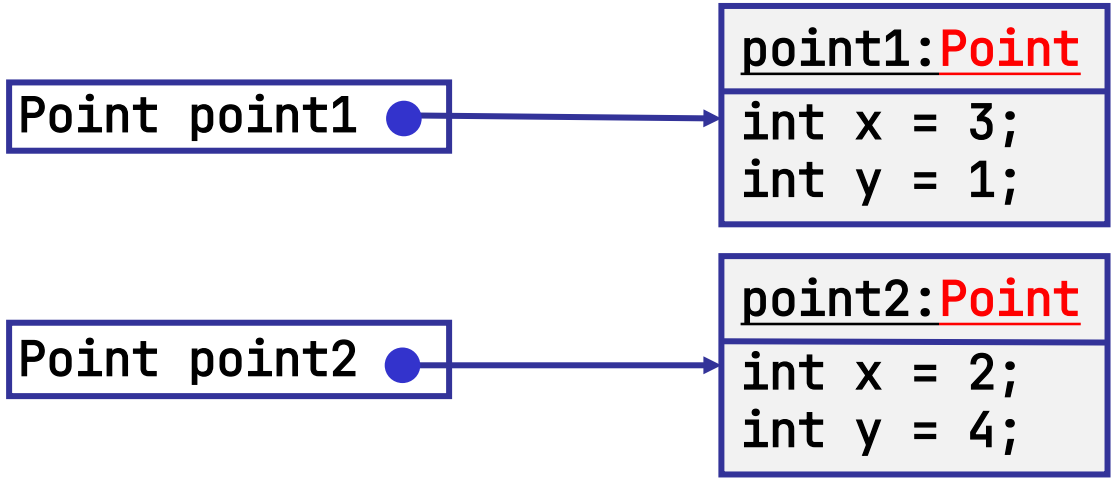
Die **häufigste** Form von Gleichheit die wir aus fachlicher Sicht benötigen!

Typgleichheit (Class Types)

- Nur der **Typ** eines Objektes steht im Vordergrund.
- Gleichheit ist bereits **dann** gegeben, wenn es sich «nur» um denselben **Typ** handelt: Die Objekte sind von derselben **Klasse**.

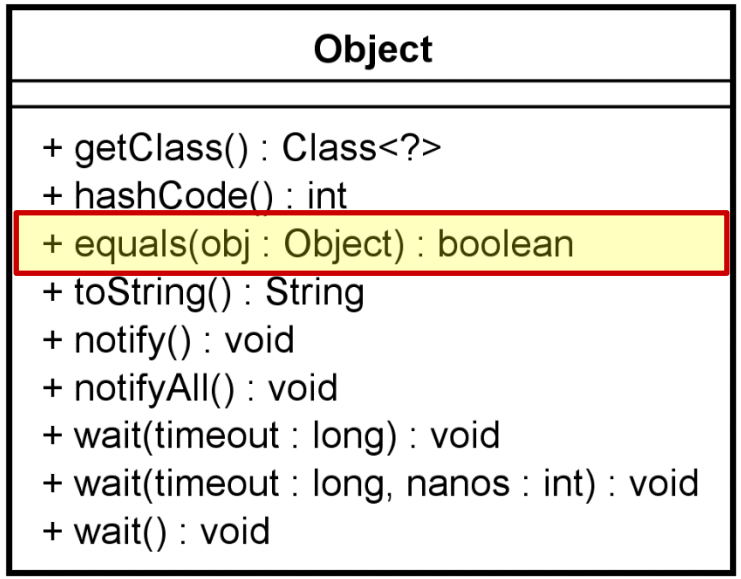
```
Point point1 = new Point(3,1);
Point point2 = new Point(2,4);
```

➔ `point1` und `point2` sind vom **gleichen** (dynamischen) **Typ**.



Wer entscheidet über die Art der Gleichheit?

Werfen wir einen Blick auf die Basisklasse `Object`



- Eine der wenigen Methoden, welche auf der Klasse **`Object`** bereits definiert ist: **`boolean equals(Object obj)`**.
- Durch das optionale **Überschreiben** (override) dieser Methode können bzw. müssen wir für jede spezialisierte Klasse (Typ) die gewünschte Art von Gleichheit **selber bestimmen**!

Verwendung von `equals()` - WICHTIG

- Viele Java-Klassen (insbesondere **Collections**) setzten voraus, dass **`equals()`** adäquat und richtig implementiert ist!
- **WICHTIG**: Wenn wir **Objekte** auf ihre Gleichheit prüfen wollen, verwenden wird **immer** die **`equals()`**-Methode!
  - **Nur** elementare Datentypen dürfen wir mit "`==`" vergleichen.
  - Weil der Gleichheitsoperator ("`==`") prüft immer auf Identität!
- Erinnerung: **`String`** ist ein Klassentyp (nicht elementarer Typ).
  Darum Strings **nie** mit "`==`", sondern **immer** mit **`equals()`** vergleichen!!

```
String name1 = "Holdrio";
String name2 = "Holdrio";
boolean never = (name1 == name2);          // FALSCH!
boolean ever  = name1.equals(name2);       // Richtig
```

Default-Implementation von `equals()` auf `Object`

- Auf **`Object`** ist die **`equals()`**-Methode wie folgt definiert:

```
public boolean equals(Object obj) {
    return (this == obj);
}
```

- Per Default wird also immer auf die einfachste, aber auch strengste Form der Gleichheit, nämlich der **Identität** (dasselbe Objekt) geprüft.
  - Nicht ganz freiwillig: **`Object`** hat ja keine nutzbaren Attribute (values)!
- Da wir in einer (grossen) Mehrheit aller Fälle aber **Value Types** benötigen, **müssen** wir diese Methode in der Regel **in den meisten unserer Klassen** überschreiben!
  - Besonders wenn diese in einer Collection gespeichert werden sollen!

`equals()`-Contract

Dokumentation von `equals()` in der Javadoc

```
public boolean equals(Object obj)
```

Indicates whether some other object is "equal to" this one.
The equals method implements an equivalence relation on non-null object references:

- It is ***reflexive***: for any non-null reference value **`x`**, **`x.equals(x)`** should return **`true`**.
- It is ***symmetric***: for any non-null reference values **`x`** and **`y`**, **`x.equals(y)`** should return **`true`** if and only if **`y.equals(x)`** returns **`true`**.
- It is ***transitive***: for any non-null reference values **`x`**, **`y`**, and **`z`**, if **`x.equals(y)`** returns **`true`** and **`y.equals(z)`** returns **`true`**, then **`x.equals(z)`** should return **`true`**.
- It is ***consistent***: for any non-null reference values **`x`** and **`y`**, multiple invocations of **`x.equals(y)`** consistently return **`true`** or consistently return **`false`**, provided no information used in equals comparisons on the objects is modified.
- For any non-null reference value **`x`**, **`x.equals(null)`** should return **`false`**.

The equals method for class Object implements the most discriminating possible equivalence relation on objects; that is, for any non-null reference values **`x`** and **`y`**, this method returns **`true`** if and only if **`x`** and **`y`** refer to the same object (**`x == y`** has the value **`true`**).
…

`equals()`-Contract - Zusammengefasst

Der `equals()`-Contract verlangt folgendes Verhalten:

- **Reflexivität** bei Vergleich mit sich selbst:
  `x.equals(x)` liefert immer `true`.
- **Symmetrie**:
  `x.equals(y)` und `y.equals(x)` liefern das gleiche Resultat.
- **Transitivität**:
  Falls `x.equals(y) == true` und `y.equals(z) == true`, dann soll auch `x.equals(z)` das Resultat `true` liefern.
- **Konsistenz** bei wiederholten Vergleichen:
  Bei unveränderten Objekten muss `equals()` immer das gleiche Resultat liefern.
- **Vergleich mit `null`**:
  `x.equals(null)` liefert `false`, **keine** `NullPointerException`!

`equals()` – Einfaches Schema für Implementation

- Falls man **`equals()`** für **Value** Types überschreibt, bietet sich folgendes Schema an:

① Test auf Identität mit "==": ➔**`true`** bei Identität sehr schnell.

② Test auf **`null`**: ➔**`false`**, weiteren Aktionen möglich.

③ Test auf Typen-Vergleichbarkeit: ➔**`false`**, wenn anderer Typ.

④ Vergleich aller relevanten Attribute der Klasse.
- Elementare Datentypen: Vergleich mit "==".
- Referenzvariablen: Vergleich an **`equals()`** weiterdelegieren!

- Hinweis: Schritt **2** und **3** können bei Verwendung von **`instanceof`** **gemeinsam** erledigt werden, weil der **`null`**-Test dort integriert ist.
  - Beispiel siehe nächstes Slide.

Beispiel: `equals()` der Klasse `Point` für Wertegleichheit

```java
@Override
public boolean equals(final Object object) {
    if (object == this) {                                   ①
        return true;
    }
    if (!(object instanceof Point)) {                       ②③
        return false;
    }
    final Point other = (Point) object;
    return (other.x == this.x) && (other.y == this.y);      ④
}
```

① (Schnell-)Test auf mögliche **Identität** (weil sehr effizient!).

② Test auf denselben **Typ** und ③ nicht `null` (kombiniert mit `instanceof` !).

④ Vergleich der relevanten Attribut**werte** für Wertegleichheit.

Beispiel: `equals()` der Klasse `Point` seit Java 17

```
@Override
public boolean equals(final Object object) {
    if (object == this) {                                  ①
        return true;
    }
    if (!(object instanceof Point p)) {                    ②③
        return false;
    }
    return (p.x == this.x) && (p.y == this.y);             ④
}
```

- Seit Java 17 erlaubt `instanceof` eine integrierte Deklaration einer Variablen, welche noch dazu einen «flow»-Scope hat.
  - flow-Scope bedeutet vereinfacht: Variable bleibt sichtbar, wo sie gebraucht wird.
- Dadurch entfällt die Zeile mit dem expliziten (down-)cast.

Beispiel: `equals()` seit Java 17 – noch kürzer!

```
@Override
public boolean equals(final Object object) {
    if (object == this) {                                  ①
        return true;
    }
    return (object instanceof Point p)                     ②③
            && (p.x == this.x)
            && (p.y == this.y);                            ④
}
```

- Da logische Verknüpfungen immer von links nach rechts ausgewertet werden, kann man auch alles in einen einzigen Ausdruck schreiben, und direkt als Rückgabewert (`boolean`) verwenden.

➔Empfohlene Form der Implementation, weil einfach und sehr kompakt!

Empfehlungen für Implementation von `equals()`

- Halten Sie sich wenn immer möglich an das empfohlene Schema.
  - Und **nicht** an irgendwelche (meist) **veralteten** Generate einer IDE oder KI!
- Wenn Sie Attribute von anderen Klassentypen (z.B. `String`) vergleichen müssen, **delegieren** Sie weiter: Also wiederum `equals()` auf diesen Attributen aufrufen!
- **Achtung**: Attribute von Klassentypen könnten ggf. auch `null` sein, das müssen Sie vorgängig prüfen, da `equals()` gemäss Contract **keine** `NullPointerException` werfen darf!
- Die Klasse `Objects` bietet dafür zur Vereinfachung eine statische Hilfsmethode an: `Objects.equals(Object a, Object b)`
- Ideal (wenn möglich): `equals()` ist nur von **finalen** (immutable) Attributen abhängig, welche auch niemals `null` sein dürfen.

Methode `hashCode()`

Einführung: `hash()`-Funktion (allgemein)

- Eine Hashfunktion ist eine Abbildung, die eine grosse Eingabe-(daten-)menge auf eine sehr viel kleinere Zielmenge (den Hashwert) abbildet.
- Populäre Einsatzgebiete:
  - Passwort-Hashes (z.B. mit **bcrypt**).
  - Datenübertragung: Sender und Empfänger prüfen, ob der Hashwert identisch ist (z.B. **MD5** oder **SHA-2**).
- Eine gute Hashfunktion liefert selbst für **kleinste** Änderungen in den Eingabedaten einen **unterschiedlichen** Hashwert.
- Die (nicht mögliche) perfekte Hashfunktion würde für alle unterschiedlichen Eingabedaten auch unterschiedliche Hashwerte liefern ➔in der Realität können aber **immer** Kollisionen auftreten!

Dokumentation von `equals()` in der JavaDoc

- Nochmal ein Blick in die (gekürzte) **`equals()`**-Dokumentation:

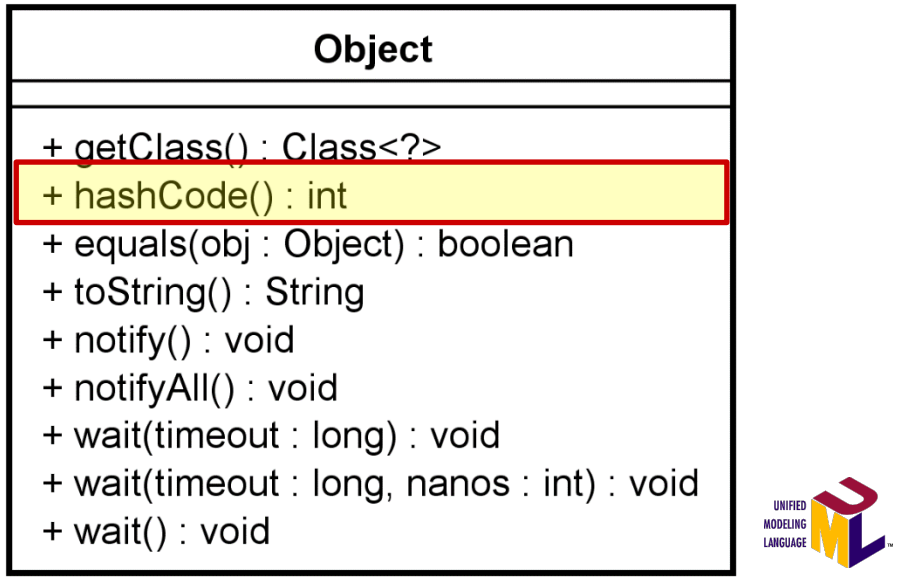
```
public boolean equals(Object obj)
```

Indicates…

Note that it is generally necessary to override the hashCode method whenever this [equals()] method is overridden, so as to maintain the general contract for the hashCode method, which states that **equal objects must have equal hash codes**.

- Die Implementation von **`equals()`** ist somit «nur die halbe Miete»: Sobald wir **`equals()`** überschreiben, **müssen** wir **auch `hashCode()`** überschreiben!
  - Damit die beiden Methoden konsistente Antworten liefern.
- Das Methodenduo **`equals()`** und **`hashCode()`** wird von vielen Collections verwendet, und ist **fundamental wichtig** für deren **korrekte** und **performante** Funktion!

`Object` definiert die Methode `hashCode()`



- **`Object`** definiert mit **`int hashCode()`** eine Hashfunktion.
- Die Methode **`hashCode()`** liefert im Einklang zu **`equals()`** quasi die «Adresse» des Objektes als Hashwert.
  - Nur **identische** Objekte liefern somit denselben **`int`**-Wert!
  - Das gilt aber nur pro einzelner Programmausführung (Laufzeit).

    ➔ Vorsicht bei persistenter Speicherung!

`hashCode()`-Contract - Zusammengefasst

Der **`hashCode()`**-Contract verlangt, dass die Implementation auf **derselben** Art von Gleichheit basiert wie **`equals()`**:

- Sind zwei Objekte im Sinne von **`equals()`** **gleich**, **müssen** auch die Hashwerte **identisch** sein!
- Sind zwei Objekte im Sinne von **`equals()`** **ungleich**, **sollten** sie idealerweise **verschiedene** Rückgabewerte liefern.
  - Das ist aber **kein** Zwang, weil das würde ja nach perfekten Hashes verlangen, was unmöglich ist.
- Konsistenz: Wird das Objekt nicht verändert, darf sich bei wiederholtem Aufruf von **`hashCode()`** auch der Hashwert nicht verändern.

`hashCode()` – Schema für Implementation

- Elementare Typen: Alle Wrapper-Typen (`Integer`, `Long`, `Float`, `Double` etc.) stellen statische Hilfsmethoden `int hashCode(…)` zur Verfügung, um die Hashwerte zu berechnen. Beispiele:
  - `Boolean.`*`hashCode`*`(true);`
  - `Integer.`*`hashCode`*`(5436);`
  - `Double.`*`hashCode`*`(2.73);`
- Klassentypen: Man delegiert die `hashCode()`-Methode weiter! Kann die Referenz `null` sein, verwendet man zur Vereinfachung die Hilfsmethode `Objects.`*`hashCode`*`(…)` welche die Referenz vorgängig auf `null` prüft.
- Setzt sich der Hashwert aus mehreren Attributen zusammen, kann man die Methode `Objects.`*`hash`*`(…)` verwenden. Sie nimmt eine **beliebige** Menge von Argumenten entgegen, prüft (filtert) diese auf `null` und berechnet daraus den Gesamthash!

Beispiel: `hashCode()` der Klasse `Point` für Wertegleichheit

- Aktuelle, einfachste Variante (seit Java 1.7 möglich):

```
@Override
public int hashCode() {
    return Objects.hash(this.x, this.y);
}
```

- In älteren Quellcodes (<= 1.6) findet man noch oft viel aufwändigere Implementationen in folgender Art (Beispiel, heute definitiv veraltet!!):

```
@Override
public int hashCode() {
     int hash = 13;
     hash = hash * 31 + this.x;
     hash = hash * 31 + this.y;
     return hash;
}
```

Der Hashwert musste damals noch «manuell» berechnet werden!

Empfehlungen zu `equals()` und `hashCode()`

- Die Methoden **`equals()`** und **`hashCode()`**:
  - **Müssen** immer **gemeinsam** und **konsistent** (basierend auf der gleichen Attributmenge) überschrieben werden!
  - Sollten in einer Vererbungshierarchie nur **ein einziges Mal** Überschreiben, und dabei möglichst finalisiert (`final`) werden!
- Wird **`equals()`** mehrfach überschrieben, oder ist schwierig korrekt zu implementieren: Design **hinterfragen**. Es kann ein **Indiz** auf **schlechte** Modellierung (typisch für eine schlecht gewählte Vererbung) sein!
- Viele moderne IDEs bieten bei der Implementation von **`equals()`** und/oder **`hashCode()`** Unterstützung an.
  - **Warnung**: Die generierten Methoden sind nicht immer korrekt oder verwenden häufig **veraltete** Techniken! Immer **kritisch hinterfragen**!

Testen von `equals()` und `hashCode()`

Empfehlungen - Testen von `equals()` und `hashCode()`

- Will man eine **`equals()`**-Methode wirklich seriös testen (was sich **sehr** lohnt), ist das leider etwas aufwändig!
  - **1**. Testfall: Identität
  - **2**. Testfall: Falsche/fremde Klasse.
  - **3**. Testfall: `null` – Werte in einzelnen Attributen.
  - **4**. – **n**. Testfall: Value-Gleichheit für jedes beteiligtes Attribut.
  - Das alles jeweils für `true` (gleich) und `false` (ungleich).
- Dieser Aufwand ist der Grund, warum es leider viel zu oft (und wider besseres Wissen) ungenügend (oder gar nicht) getestet wird!
  - Eine Fehlersuche gestaltet sich dann aber umso aufwändiger!

➔ Seriöse Tests dieser beiden Methoden **lohnen** sich deshalb, und sie bergen ein hohes Qualitätspotential!

Empfehlungen – EqualsVerifier-Library

- Eine Wunderwaffe zum Testen von **`equals()`** und **`hashCode()`**:
  EqualsVerifier – siehe http://jqno.nl/equalsverifier/
  - Einfach in der Anwendung und ausgezeichnet gut dokumentiert!
- Beispiel für die **`Point`**-Klasse, als Unit Test:

```
@Test
public void testEqualsContract() {
    EqualsVerifier.forClass(Point.class).verify();
}
```

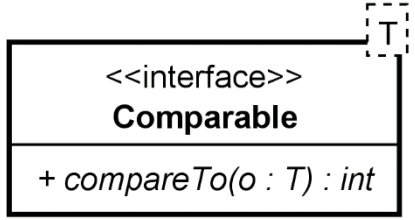
- Damit ist zumindest der **formale `equals()`**-Contract geprüft!
  - Reflexivität, Symmetrie, Transitivität, Konsistenz… - all das wird nämlich noch seltener getestet, ist aber umso wichtiger!

➔ **Nutzen Sie dieses geniale Tool unbedingt!** (Diese Empfehlung ist ein Befehl!)
  - Im OOP-Template-Projekt als Dependency bereits enthalten!

Interface `Comparable<T>`

Kleiner? oder Grösser? - Interface `Comparable<T>`

- Genauso häufig wie auf «Gleichheit», möchten wir auch auf «kleiner als» ("`<`") und «grösser als» ("`>`") prüfen können.
  - z.B. um das grösste oder kleinste Datenelement zu finden.
- Auch dafür gibt es in Java ein Interface:



- Das Interface **`Comparable`** erlaubt es beliebigen Klassen, sich mit anderen Objekten **desselben** Typs «vergleichbar» zu machen.
  - Im Sinne von **kleiner**, **gleich** (!) oder **grösser**.
- Die Implementation dieses Interfaces ist eine wichtige Grundlage, dass Objekte in den JCF-Datenstrukturen **sortiert** werden können.

`compareTo()`-Contract

- Auch **`compareTo()`** liegt wieder ein «Vertrag» zu Grunde:

```
public int compareTo(T obj)
```

Compares this object with the specified object for order. Returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object.

- Etwas strukturierter zusammengefasst:
  - **`this < other`** liefert einen negativen Wert (typisch **`-1`**).
  - **`this == other`** liefert **`0`**.
  - **`this > other`** liefert einen positiven Wert (typisch **`+1`**).
  - Analog zur **`equals()`**-Methode ist auch hier **Symmetrie** und **Transitivität** verlangt.
- **Empfehlung**: **`compareTo()`** und **`equals()`** sollten Gleichheit möglichst analog behandeln! (kein Zwang, aber häufig sinnvoll)

`compareTo()`-Methode - Schema für Implementation

Es empfiehlt sich folgendes Schema einzuhalten:

1. Test auf Identität mit "`==`" (➔ `0` bei Identität, schnell)
2. Schrittweiser Vergleich aller relevanter Attribute:
   - **Elementare Datentypen**: Vergleich mit Operatoren "`==`", "`>`" und "`<`" möglich, aber **besser** `compare(…)` auf den jeweiligen Wrapper-Typen (`Integer`, `Long`, `Double` etc.) verwenden.
   - **Klassentypen**: Vergleich jeweils an die jeweils eigene `compareTo()`-Methode weiter delegieren.

- Wichtige Hinweise:
  - Beim Vergleich mehrerer Attribute ist die Reihenfolge natürlich relevant!
  - Bei Attributen von Klassentypen muss man ggf. `null`-Werte berücksichtigen!

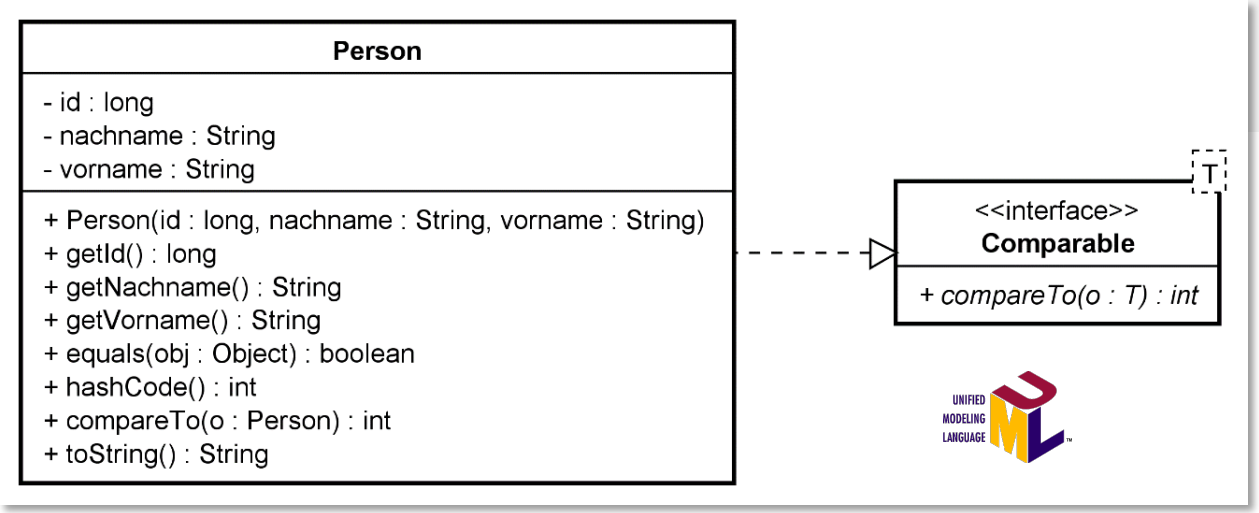
Beispiel: `compareTo()` von `Person`

- Vergleicht Personen auf Basis ihrer Personalnummer (**`id`**):

```java
public final class Person implements Comparable<Person> {

    privat final long id;
    ...

    @Override
    public int compareTo(Person other) {
        return Long.compare(this.id, other.id);
    }
    ...
}
```

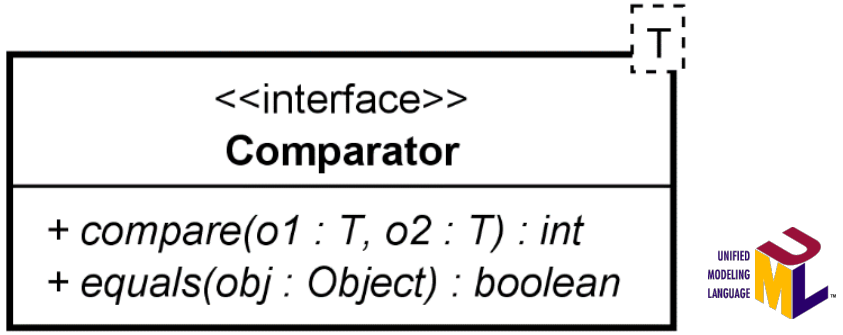


Natürliche Ordnung vs. spezielle Ordnung

Natürlich Ordnung vs. spezielle Ordnung

- **Natürliche** Ordnung:
  Eine Klasse implementiert das Interface **`Comparable`** und definiert damit eine einzige, «natürliche» Ordnung. Beispiele:
  - Temperaturen nach Wert aufsteigend.
  - Namen alphabetisch aufsteigend: Nach Nachnamen, Vornamen.
- **Spezielle** Ordnung:
  Wollen wir Objekte nach verschiedensten Kriterien sortieren, implementieren wir **eigenständige**, kleine Klassen, welche das Interface **`Comparator`** implementieren.
  - Jeder Komparator steht dann für eine spezielle Ordnung.

Interface `Comparator<T>`

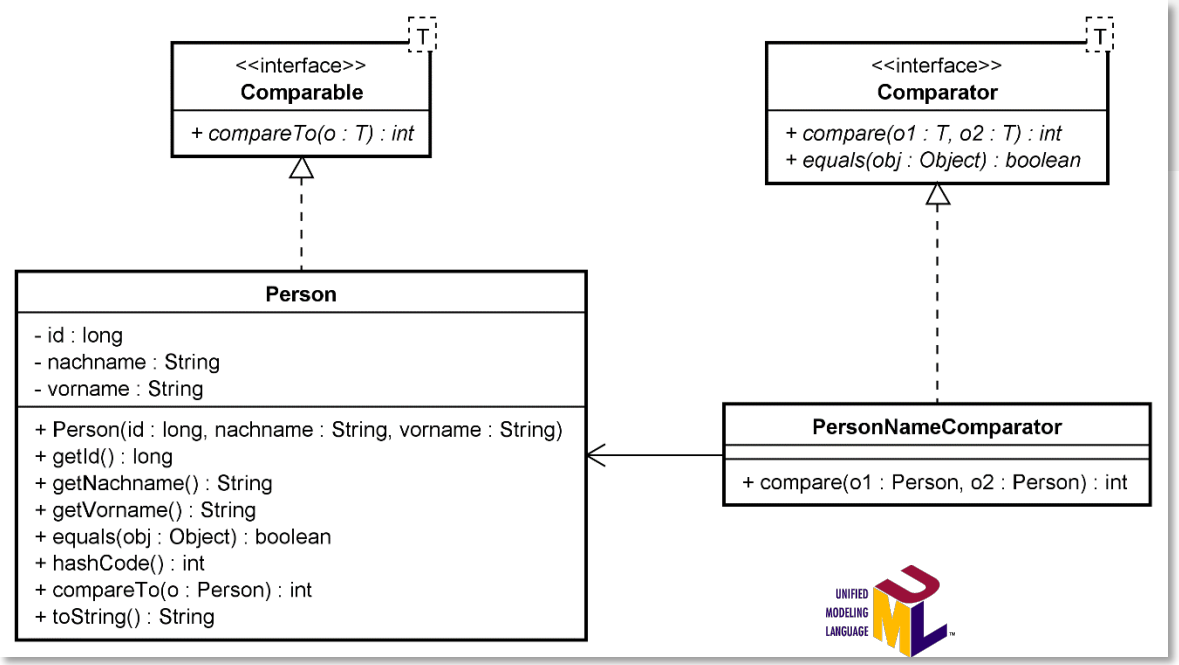


Das Interface spezifiziert folgende Methoden:

- `int compare(T o1, T o2)`
  Vergleicht zwei Objekte in analoger Semantik wie **`Comparable`**.
- `boolean equals(Object obj)`
  Soll so implementiert werden, dass Comparatoren mit **gleicher** Sortierlogik auch als "gleich" erkannt werden (für Optimierungen).
  ➔ einer der seltenen Fälle, bei welchem es ausreicht, "nur" auf den Klassentyp (Class Type) zu vergleichen.
- **Achtung**: Obschon im Interface nicht definiert, werden wir somit implizit gezwungen, auch die Methode **`hashCode()`** entsprechend zu implementieren! ➔ **`equals()`**-Contract!

Beispiel: Comparator<Person> auf Nachname und Vorname

```java
public class PersonNameComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        int compare = p1.getNachname().compareTo(p2.getNachname());
        if (compare == 0) {
            compare = p1.getVorname().compareTo(p2.getVorname());
        }
        return compare;
    }
}
```



Zusammenfassung

- Verschiedene Arten von Gleichheit (Entity, Value, Class).
- Die verschiedenen Verträge (contracts):
  `equals()`, `hashCode()` und `compareTo(…)`
- Korrekte Implementation und Test dieser Methoden.
- `Comparable<T>`-Interface für die Definition der natürlichen Ordnung einer Klasse.
- `Comparator<T>`-Interface für beliebige, spezielle Ordnungen.
- All diese Verträge und Regeln erscheinen auf den ersten Blick vielleicht etwas kompliziert und umfangreich. Wenn man sie aber verstanden hat und seriös implementiert, kann man das Potential der verschiedenen Datenstrukturen von Java erst richtig ausschöpfen!

Fragen?